

LIBRARY MANAGEMENT SYSTEM

PROJECT REPORT

Technology: Python

Application Type: Console-Based Library Management System

Author: Khilen Raj

Detailed explanation of the supplied Python project, including architecture, classes, functions, data persistence, workflows, validation, testing, limitations and future enhancements.

1. Project Overview

The Library Management System is a console-based Python application for managing books, members, and book issue/return operations. The supplied project lists features including adding, removing, updating and searching books; registering and removing members; issuing and returning books; viewing books and members; and persistent JSON storage.

2. Objectives

The system is designed to maintain library book inventory, member records and active loans; automatically calculate a 14-day loan period; calculate a Rs. 5 per-day late fine; provide a menu-driven interface; and preserve information using books.json and members.json.

3. Technologies and Modules

The project uses Python, the json module for JSON serialization, os for checking file existence, and datetime/timedelta for due-date and late-return calculations. The constants define the two JSON filenames and LOAN_PERIOD_DAYS = 14.

4. Data Persistence

load_data(filename) checks whether a file exists. If it exists, json.load() reads the contents into Python data; otherwise an empty dictionary is returned. save_data(filename, data) writes data with json.dump(..., indent=4). This allows records to survive program restarts.

5. Book Class

Book represents a library book. Its constructor stores book_id, title, author, copies and available. Total copies represent the library's ownership, while available represents copies that can currently be issued. to_dict() prepares the object for JSON storage and from_dict() reconstructs a Book object from stored data.

6. Member Class

Member represents a registered user. It stores member_id, name and email. borrowed_books is a dictionary mapping a book ID to its due-date string. to_dict() serializes the member and from_dict() restores the saved member state.

7. Library Class — Core System

Library is the central management class. It maintains books and members dictionaries. During initialization it calls _load(), which reads both JSON files and reconstructs Book and Member objects. _save() converts current objects to dictionaries and writes them back to the JSON files.

8. Book Management

add_book() rejects duplicate IDs, creates a Book, saves it and confirms success. remove_book() deletes an existing book. search_book() performs a case-insensitive search against title and author. list_books() prints each book's ID, title, author and available/total copies.

9. Member Management

`add_member()` rejects duplicate member IDs, creates a Member and saves it. `remove_member()` deletes an existing member. `list_members()` displays each member's ID, name, email and currently borrowed book IDs.

10. Issue Book Process

`issue_book()` retrieves the requested book and member. It validates both IDs, checks that a copy is available and prevents the same member from borrowing the same book twice. It then calculates a due date 14 days from the current date, records the loan, decreases available copies and saves the updated data.

11. Return Book and Fine

`return_book()` validates the book and member and confirms that the member actually borrowed the book. It removes the loan record, increases available copies and saves. The due-date string is converted to a datetime. If the current date is later than the due date, days late are multiplied by Rs. 5 to calculate the fine.

12. Console Menu

`main()` creates a Library object and repeatedly displays a menu. Options 1–9 perform the available book/member/loan operations, while 0 exits. User input is collected with `input()` and passed to the relevant Library method. Invalid menu choices produce an error message.

13. Complete System Workflow

Program start → Library object created → JSON data loaded → menu displayed → user selects an operation → validation and business logic run → data is saved → result is displayed → menu appears again. Issue and return operations update both the member loan record and book availability.

14. Data Model

Book: `book_id`, title, author, copies, available. Member: `member_id`, name, email, `borrowed_books`. A loan record is represented inside `borrowed_books` as `book_id` mapped to a due-date string.

15. Object-Oriented Programming Concepts

The project uses classes and objects, encapsulates book/member data and behavior, uses a static `from_dict()` constructor-style method, and separates responsibilities among Book, Member, Library, persistence helpers and the main user-interface function.

16. Error Handling and Validation

The supplied code checks duplicate IDs, missing books or members, unavailable books, repeated borrowing, invalid return records and invalid menu choices. The supplied code does not show broad exception handling for malformed JSON, file failures, or non-numeric input entered where `int()` is expected.

17. Testing Scenarios

Useful demonstrations include adding and listing a book; trying a duplicate book ID; registering a member; searching by title/author; issuing an available book; attempting to issue an unavailable book; attempting

duplicate borrowing; returning a book; testing a late return and fine; and restarting the program to verify JSON persistence.

18. Advantages

The project provides a simple interface, object-oriented structure, persistent storage, book searching, inventory tracking, member tracking, automatic due dates and automatic late-fine calculation.

19. Limitations and Future Enhancements

The supplied code could be extended with stronger input/file exception handling, protection against removing books with active loans, protection against removing members with borrowed books, a separate book-update operation, richer loan history, authentication, a relational database, a graphical/web interface and fine/payment history.

20. Conclusion

The supplied Library Management System demonstrates a practical Python application combining object-oriented programming, dictionaries, JSON persistence, date calculations and a command-line interface. Book and Member model the main entities, while Library coordinates inventory, membership and borrowing operations.

Key Function Reference

Function	Purpose
load_data(filename)	Loads JSON data if the file exists.
save_data(filename,data)	Writes data to JSON.
Book.to_dict() / from_dict()	Converts between Book objects and dictionaries.
Member.to_dict() / from_dict()	Converts between Member objects and dictionaries.
Library._load() / _save()	Loads and persists books and members.
add_book() / remove_book()	Manages book records.
search_book() / list_books()	Searches and displays books.
add_member() / remove_member()	Manages member records.
list_members()	Displays member and active-loan information.
issue_book()	Validates and records a loan.
return_book()	Processes a return and calculates a late fine.
main()	Runs the console menu and user interaction.

Project Links

The following GitHub resources are associated with this project:

1. Project Report (GitHub)

https://github.com/nikhil-kumar2005/upskillcampus/blob/main/Library_Management_System_Nikhil_USC_UCT.pdf

2. Python Source Code (GitHub)

<https://github.com/nikhil-kumar2005/upskillcampus/blob/main/LibraryManagementSystem.py>